

CSE-719(Distributed and Cloud Computing)

Dr. Atiqur Rahman

ড. আতিকুর রহমান

Ph.D.(CQUPT, China), MS.Engg.(CU), B.Sc.(CU)

Associate Professor

Department of Computer Science and Engineering

University of Chittagong

Lecture 3: Mapreduce and Hadoop

What is MapReduce?

- MapReduce is **a programming framework that allows us to perform distributed and parallel processing on large data sets in a distributed environment.** MapReduce consists of two distinct tasks – Map and Reduce. As the name MapReduce suggests, the reducer phase takes place after the mapper phase has been completed. MapReduce can be written in **Java, Python**, etc. The choice of a programming language depends on programmer i.e. how comfortable you are with a particular language. Though Hadoop is written in Java but you can write MapReduce in any language you feel comfortable.
- **Hadoop:** Open source MapReduce software platform
- Terms are **borrowed** from **Functional Language** (e.g., **Lisp**)
- Hadoop is a framework that uses distributed storage and parallel processing to store and manage big data.

Hadoop

- **Hadoop**: Open source MapReduce software platform
- Distributes data and processing across many nodes
- Processes the data locally at each node
- Transparent fault tolerance through
 - Automatic data duplication
 - Automatic detection and restarting of failing nodes
- Automatically provides framework for developing MapReduce applications
 - Handles mapping and reducing logistics
 - Programmer just provides custom functionality
- Currently takes custom functionality in Java and Python
- Uses an open source Eclipse plug-in to interface with Hadoop

What is MapReduce(Cont.)?

Sum of squares: (Lisp Code)

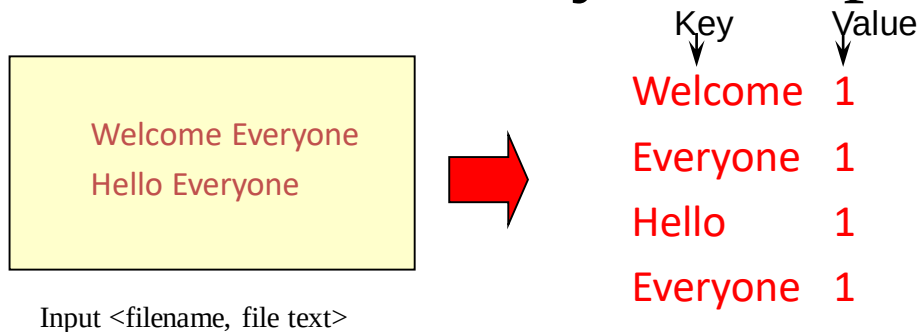
- (map square '(1 2 3 4))
 - Output: (1 4 9 16)

[processes each record sequentially and independently]
- (reduce + '(1 4 9 16))
 - (+ 16 (+ 9 (+ 4 1)))
 - Output: 30

[processes set of all records in batches]
- Let's consider a sample application: **Wordcount**
 - You are given a huge dataset (e.g., Wikipedia dump or all of Shakespeare's works) and asked to list the count for each of the words in each of the documents therein

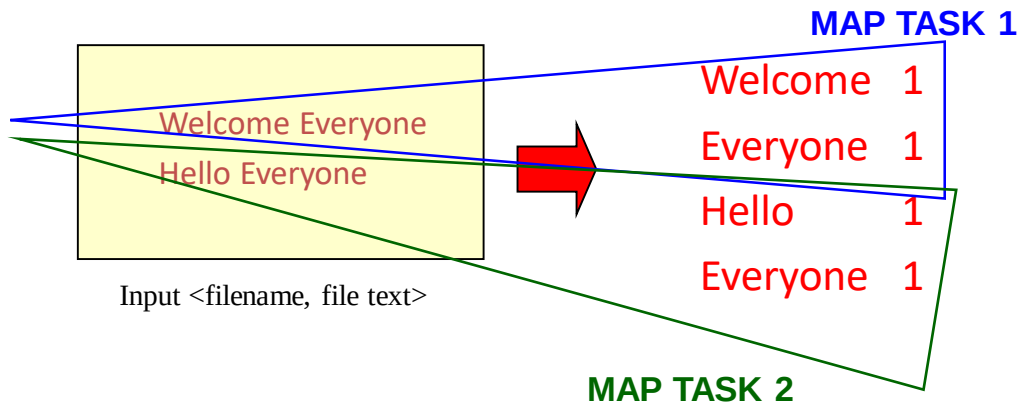
Map

- Process individual records to generate intermediate key/value pairs.



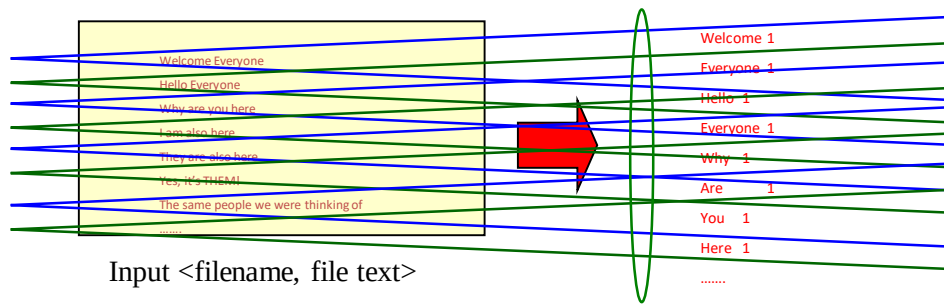
Map

- **Parallely** Process individual records to generate intermediate key/value pairs.



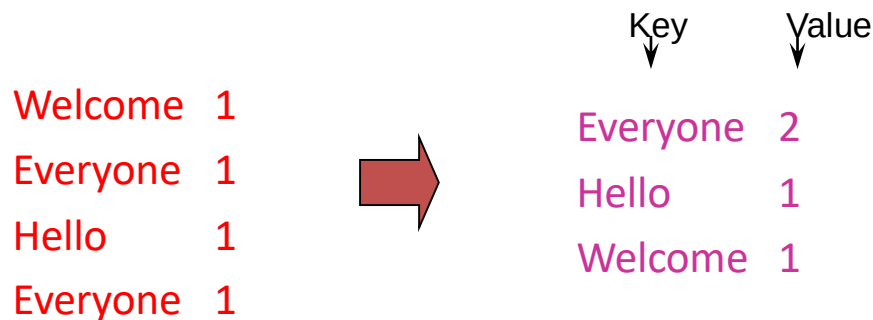
Map

- **Parallely** Process **a large number** of individual records to generate intermediate key/value pairs.



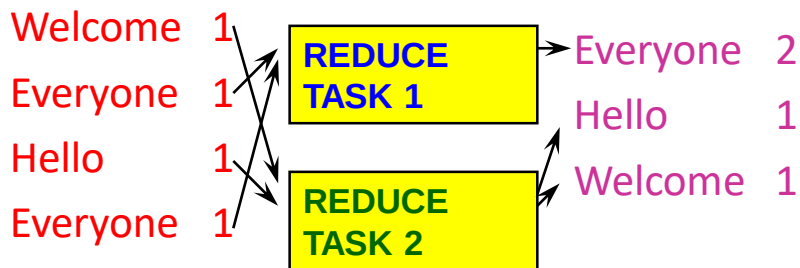
Reduce

- Reduce processes and merges all intermediate values associated per key



Reduce

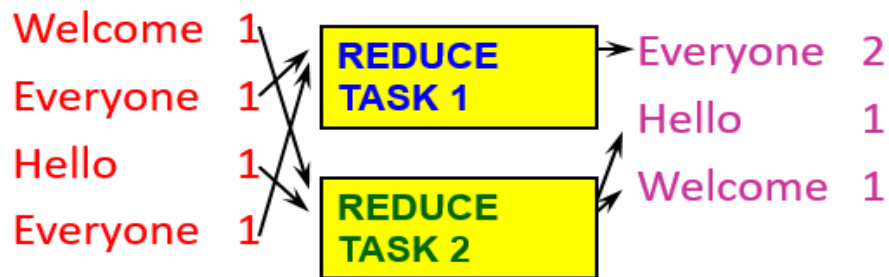
- Each key assigned to one Reduce
- Parallelly Processes and merges all intermediate values by partitioning keys



- Popular: *Hash partitioning*, i.e., key is assigned to reduce # = $\text{hash}(\text{key}) \% \text{number of reduce servers}$

Reduce(Cont.)

- Popular: **Hash partitioning**, i.e., key is assigned to reduce # = $\text{hash}(\text{key}) \% \text{number of reduce servers}$



- Partitioning is **based on a function of one or more columns (the hash partitioning keys) in each record**. The hash partitioner examines one or more fields of each input record (the hash key fields). **Records** with the **same values** for all **hash key** fields are assigned to the **same processing node**.

Hadoop Code - Map

```
public static class MapClass extends MapReduceBase      implements
Mapper<LongWritable, Text, Text, IntWritable> {
    private final static IntWritable one =
        new IntWritable(1);
    private Text word = new Text();

    public void map( LongWritable key, Text value,
        OutputCollector<Text, IntWritable> output, Reporter reporter)
        throws IOException {
        String line = value.toString();
        StringTokenizer itr = new StringTokenizer(line);
        while (itr.hasMoreTokens()) {
            word.set(itr.nextToken());
            output.collect(word, one);
        }
    }
}

// Source: http://developer.yahoo.com/hadoop/tutorial/module4.html#wordcount
```

Hadoop Code - Reduce

```
public static class ReduceClass extends MapReduceBase implements
Reducer<Text, IntWritable, Text, IntWritable> {
    public void reduce(
        Text key,
        Iterator<IntWritable> values,
        OutputCollector<Text, IntWritable> output,
        Reporter reporter)
        throws IOException {
        int sum = 0;
        while (values.hasNext()) {
            sum += values.next().get();
        }
        output.collect(key, new IntWritable(sum));
    }
} // Source: http://developer.yahoo.com/hadoop/tutorial/module4.html#wordcount
```

Hadoop Code - Driver

```
// Tells Hadoop how to run your Map-Reduce job
public void run (String inputPath, String outputPath)
    throws Exception {
    // The job. WordCount contains MapClass and Reduce.
    JobConf conf = new JobConf(WordCount.class);
    conf.setJobName("mywordcount");
    // The keys are words
    (strings) conf.setOutputKeyClass(Text.class);
    // The values are counts (ints)
    conf.setOutputValueClass(IntWritable.class);
    conf.setMapperClass(MapClass.class);
    conf.setReducerClass(ReduceClass.class);
    FileInputFormat.addInputPath(
        conf, newPath(inputPath));
    FileOutputFormat.setOutputPath(
        conf, new Path(outputPath));
    JobClient.runJob(conf);
}
```

// Source: <http://developer.yahoo.com/hadoop/tutorial/module4.html#wordcount>

Some Applications of MapReduce

Distributed Grep:

- Input: large set of files
- Output: lines that match pattern
- Map – *Emits a line if it matches the supplied pattern*
- Reduce – *Copies the intermediate data to output*

N.B: Grep is used to search for a String pattern in file. MapReduce is a programming model which takes care.

Some Applications of MapReduce

(2)

Reverse Web-Link Graph

- Input: Web graph: tuples (a, b) where (page a $\rightarrow$ page b)
- Output: For each page, list of pages that link to it
- Map – *process web log and for each input $\langle \text{source}, \text{target} \rangle$, it outputs $\langle \text{target}, \text{source} \rangle$*
- Reduce - *emits $\langle \text{target}, \text{list}(\text{source}) \rangle$*

Reverse Web-Link Graph — **The map function outputs $\langle \text{target}, \text{source} \rangle$ pairs for each link to a target URL found in a page named “source”.** The reduce function concatenates the list of all source URLs associated with a given target URL and emits the pair: $\langle \text{target}, \text{list}(\text{source}) \rangle$.

Some Applications of MapReduce

(3)

Count of URL access frequency

- Input: Log of accessed URLs, e.g., from proxy server
- Output: For each URL, % of total accesses for that URL
- Map – *Process web log and outputs $\langle \text{URL}, 1 \rangle$*
- Multiple Reducers - *Emits $\langle \text{URL}, \text{URL_count} \rangle$*
(So far, like Wordcount. But still need %)
- Chain another MapReduce job after above one
- Map – *Processes $\langle \text{URL}, \text{URL_count} \rangle$ and outputs $\langle 1, (\langle \text{URL}, \text{URL_count} \rangle) \rangle$*
- 1 Reducer – Sums up *URL_count's* to calculate overall_count.
Emits multiple $\langle \text{URL}, \text{URL_count}/\text{overall_count} \rangle$

Some Applications of MapReduce

(4)

Map task's output is sorted (e.g., quicksort)

Reduce task's input is sorted (e.g., mergesort)

Sort

- Input: Series of (key, value) pairs
- Output: Sorted <value>s
- Map – $\langle \text{key}, \text{value} \rangle \rightarrow \langle \text{value}, _ \rangle$ (*identity*)
- Reducer – $\langle \text{key}, \text{value} \rangle \rightarrow \langle \text{key}, \text{value} \rangle$ (*identity*)
- Partitioning function – partition keys across reducers based on ranges (can't use hashing!)
 - Take data distribution into account to balance reducer tasks

Programming MapReduce

Externally: For user

1. Write a Map program (short), write a Reduce program (short)
2. Specify number of Maps and Reduces (parallelism level)
3. Submit job; wait for result
4. Need to know very little about parallel/distributed programming!

Internally: For the Paradigm and Scheduler

1. Parallelize Map
2. Transfer data from Map to Reduce
3. Parallelize Reduce
4. Implement Storage for Map input, Map output, Reduce input, and Reduce output

(Ensure that no Reduce starts before all Maps are finished. That is, ensure the **barrier** between the Map phase and Reduce phase)

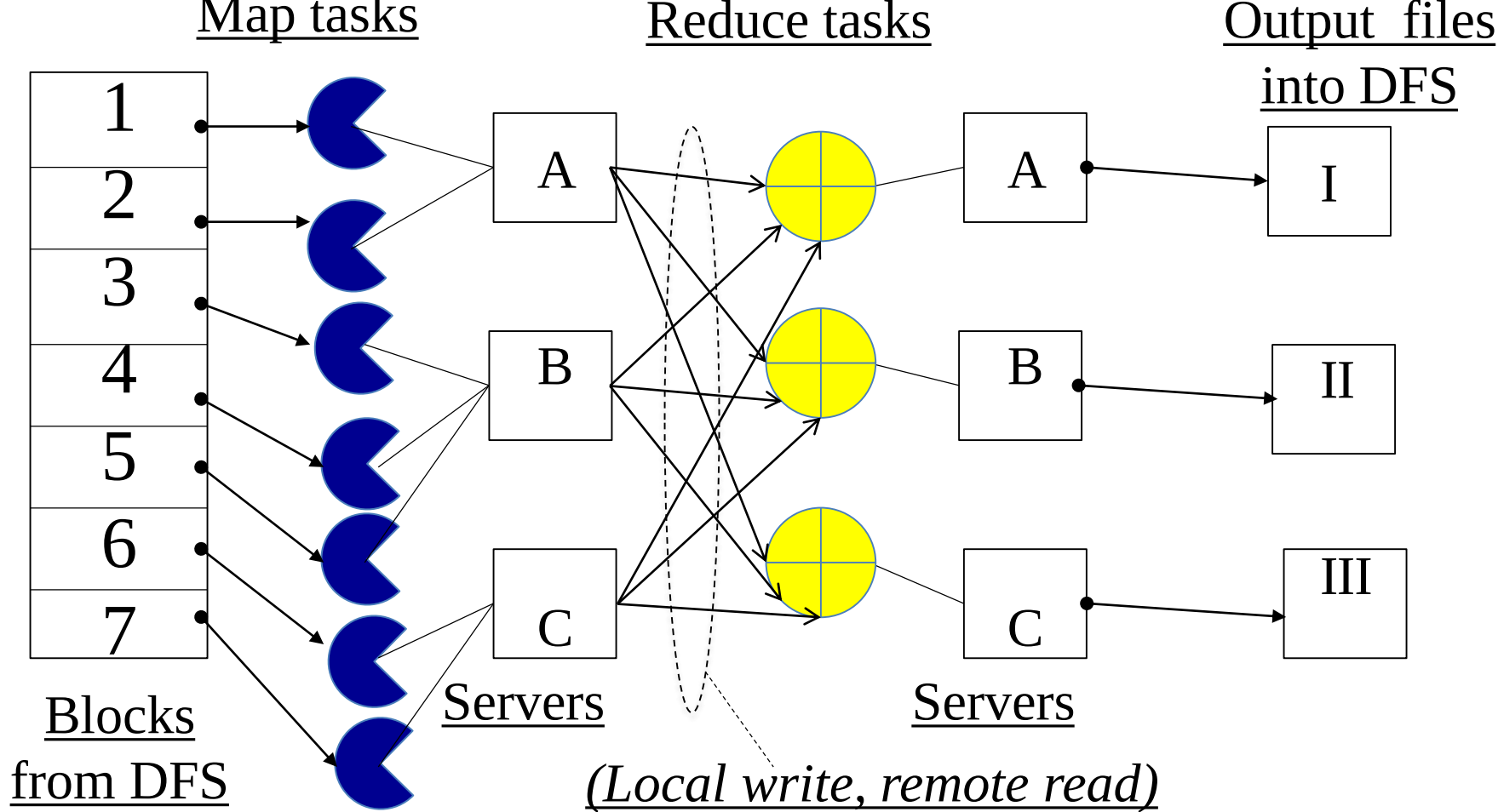
Inside MapReduce

For the cloud:

1. Parallelize Map: **easy!** each map task is independent of the other!
 - All Map output records with same key assigned to same Reduce
2. Transfer data from Map to Reduce:
 - All Map output records with same key assigned to same Reduce task
 - use **partitioning function, e.g., $\text{hash}(\text{key})\% \text{number of reducers}$**
3. Parallelize Reduce: **easy!** each reduce task is independent of the other!
4. Implement Storage for Map input, Map output, Reduce input, and Reduce output
 - Map input: from **distributed file system**
 - Map output: to local disk (at Map node); uses **local file system**
 - Reduce input: from (multiple) remote disks; uses local file systems
 - Reduce output: to distributed file system

local file system = Linux FS, etc.

distributed file system = GFS (Google File System), HDFS (Hadoop Distributed File System)



Resource Manager (assigns maps and reduces to servers)

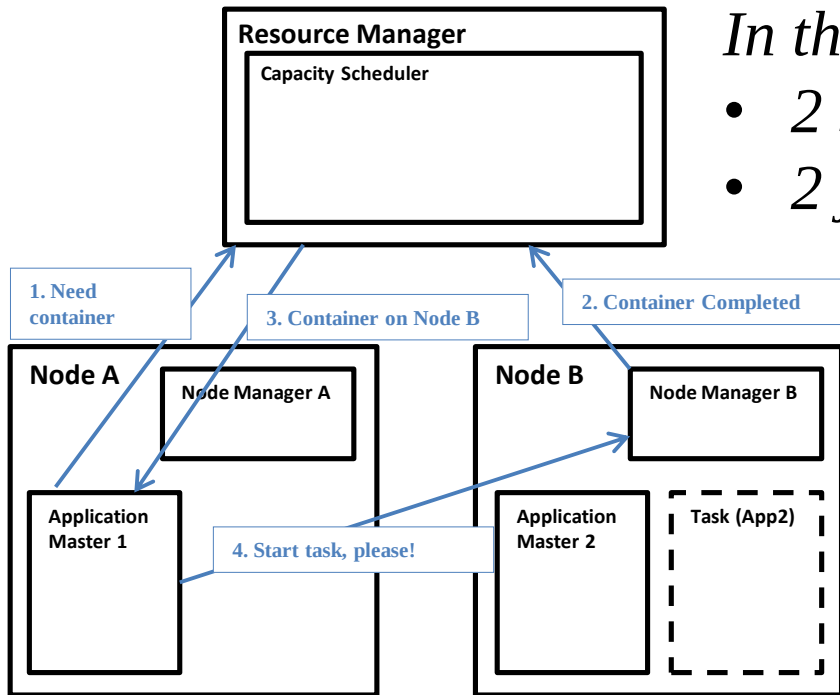
The YARN Scheduler

- Used in Hadoop 2.x +
- YARN = Yet Another Resource Negotiator
- Treats each server as a collection of *containers*
 - Container = fixed CPU + fixed memory
- Has 3 main components
 - Global *Resource Manager (RM)*
 - Scheduling
 - Per-server *Node Manager (NM)*
 - Daemon and server-specific functions
 - Per-application (job) *Application Master (AM)*
 - Container negotiation with RM and NMs
 - Detecting task failures of that job

YARN: How a job gets a container

In this figure

- 2 servers (A, B)
- 2 jobs (1, 2)



Fault Tolerance

- Server Failure
 - NM heartbeats to RM
 - If server fails, RM lets all affected AMs know, and AMs take action
 - NM keeps track of each task running at its server
 - If task fails while in-progress, mark the task as idle and restart it
 - AM heartbeats to RM
 - On failure, RM restarts AM, which then syncs up with its running tasks
- RM Failure
 - Use old checkpoints and bring up secondary RM
- Heartbeats also used to piggyback container requests
 - Avoids extra messages

Slow Servers

Slow tasks are called **Stragglers**

- The slowest task slows the entire job down (why?)
- Due to Bad Disk, Network Bandwidth, CPU, or Memory
- Keep track of “progress” of each task (% done)
- Perform proactive backup (replicated) execution of straggler task: task considered done when first replica complete. Called **Speculative Execution**.

Locality

- Locality
 - Since cloud has hierarchical topology (e.g., racks)
 - GFS/HDFS stores 3 replicas of each of chunks (e.g., 64 MB in size)
 - Maybe on different racks, e.g., 2 on a rack, 1 on a different rack
 - Mapreduce attempts to schedule a map task on
 - a machine that contains a replica of corresponding input data, or failing that,
 - on the same rack as a machine containing the input, or failing that,
 - Anywhere

Mapreduce: Summary

- Mapreduce uses parallelization + aggregation to schedule applications across clusters
- Need to deal with failure
- Plenty of ongoing research work in scheduling and fault-tolerance for Mapreduce and Hadoop